MECHTRON / SFWRENG 3K04: Software Development

Project Deliverable 2 – Fall 2026

McMaster University

Due Date: Friday, November 6th, 2026 11:59 PM

This handout applies to both parts of the project. Each sub-team will make their own submission as explained below.

1 Deliverable 2 Purpose

The purpose of this second deliverable is to move from *what* your system must do to *how* it will be built. Building on the requirements and models from Deliverable 1, you will write a precise specification of the system's required behaviour, design a modular architecture with documented interfaces that showcases information hiding, and produce a working implementation of your design that you will run live for the TAs during your demonstration. This deliverable is where your design decisions become concrete and defensible. Keep building!

2 Grading

This deliverable is **10%** of your final course grade. You will be assessed in two ways: a technical demonstration to the TAs during your lab (worth 70%) and the evaluation of your submission file(s) (worth 30%).

Deliverable 2 will be demo'ed during your lab session the week of **November 2nd, 2026**. The submission files will be due at the end of the week (**Friday, November 6th, 2026 11:59 PM**).

3 Technical Demonstration (70%)

The students will walk through the precise specification and the modular design, and run a live demonstration of their working solution that the TAs can interact with directly. You will be expected to show, in your running solution and your code, how your decomposition and modularity have been implemented, and the TAs will ask you to trigger specific behaviours and to justify design decisions on the spot. Every student must be able to speak to every module, interface, and design choice, including parts they did not personally write. The TAs will choose who answers.

What will the TAs look for:

- The precise specification is unambiguous and traceable back to the Deliverable 1 requirements, and the team can explain it in terms of the 4-variable model (monitored / controlled variables, the software's inputs and outputs, and the required relations between them).
- The team can show how their decomposition and modularity have been implemented: where the module boundaries live in the code, how modules connect through their interfaces, and how information hiding is reflected in the implementation.
- Module interfaces are clear (exported operations, their inputs/outputs, and their semantics) and the implementation matches them.

- The running solution can be interacted with by the TAs and demonstrates the key scenarios from Deliverable 1 (e.g., ADC: authenticate ->dispense ->log; Monitoring: receive alarm ->display; plus an off-normal path such as a hardware fault or communication loss) on request.
- The solution is clearly the team's own work, and every member understands the modules across the whole system.
- The team can absorb an on-the-spot change (a new anticipated change or a revised requirement) and reason about its effect on the design.

4 Submission File(s) (30%)

A well-structured design document (with the diagrams embedded). This is the evidence the TAs mark the demonstration against, and it should read as professional documentation. While the full document is due after your technical demonstration, you should have it ready (about 95% done) during the demonstration as the TAs will ask you questions about it. Note that your working implementation is assessed live during the demonstration, not submitted as a file.

Structure of the design document:

You should use your Deliverable 1 document as a base.

- **Introduction and context:** Introduction to the document and a short summary of what changed since Deliverable 1 (e.g., corrected or clarified requirements). Reference your Deliverable 1 requirement/story IDs.
- **Precise specification:** a refinement of your Deliverable 1 requirements into precise, unambiguous specifications of the required behaviour. Use the 4-variable model to frame the system boundary (monitored and controlled variables, the software's inputs and outputs, and the NAT / REQ relations). For the safety-critical functions, specify behaviour precisely (e.g., pre-conditions, post-conditions, and state invariants).
- **Modular design and interfaces:** decompose your system into modules in a way that showcases information hiding such that each module hides an anticipated change (Parnas style). For each module, state its responsibility and briefly document its interface: the exported operations, their inputs and outputs, and their semantics (assumptions, effects, exceptions). Include the external ADC <– >Monitoring System interface (what your part provides, what it requires, and how it behaves when the other side is unavailable).
- **Implementation notes:** a short description of how your design maps to the code such as which modules/classes implement which part, and how the models from Deliverable 1 (state machines, activity flows) were realized in code. You will show and run this code during your demonstration.
- **Traceability table:** extend your Deliverable 1 IDs forward such as requirement ID ->precise specification ->module ->code (file/class), so the whole set of views stays connected.
- **Assumptions and decisions:** summary of every new assumption and design decision made since Deliverable 1, and the reasoning behind them. Continue to include accessibility and ethical considerations (e.g., patient safety, privacy of medication and patient-related records, controlled/authorized access, etc.)

5 Tips for Success

- Choose module boundaries around things that are likely to change (hardware interfaces, the communication protocol, the local UI, storage), not around steps in a process. This is how you showcase information hiding.
- Keep your interface documentation and your code in sync. A mismatch between the documented interface and the running implementation is the fastest way to lose demonstration marks.
- Make sure your solution is ready to run and interact with at the start of your demonstration (set up on the machine you will present from), not something you scramble to launch. The TAs will ask you to run specific scenarios and to interact with your solution directly.
- Be ready to open your code and point to where your decomposition and modularity live. Organize your code so module boundaries and interfaces are easy to find.
- Keep tracing forward from your Deliverable 1 requirement/story IDs. Traceability from requirement to specification to module to code is what makes your design defensible under questioning.
- Rehearse as a sub-team before the demo so any member can defend any module, interface, or design decision. Seek clarification when needed from the TAs and/or *ReQuest*.

6 Submission Guidelines

Each sub-team will see an assignment folder to upload the design document on Avenue. Allow yourself enough time before the deadline in case of any technical difficulties.